



Rapport: Cheminement des Actions du Site

Épicerie BIO NATURE - Documentation Technique

Généré: 23 février 2026



Table des Matières

1. Architecture Générale
2. Action 1: Accueil (GET /)
3. Action 2: Produits (GET /products)
4. Action 3: Ajouter au Panier (POST /products/add/{id})
5. Action 4: Retirer du Panier (POST /products/remove/{id})
6. Action 5: Vider le Panier (POST /clear-cart)
7. Action 6: Contact (GET /contact)
8. Action 7: Envoyer Message (POST /send-contact)
9. Action 8: Valider Commande (GET /validate-order)
10. Modèles de Données
11. Résumé Technique



1. Architecture Générale

Le site utilise le framework **Slim Framework 4**, un micro-framework PHP moderne basé sur les PSR (PHP Standards Recommendations).

Structure du Projet

Fichier d'entrée:`index.php`

Initialise l'application Slim, enregistre toutes les routes et lance l'application.

Bootstrap:`app/bootstrap.php`

Enregistre l'autoloader PSR-4 personnalisé pour charger les classes de l'application.

Contrôleurs:`app/Controllers/`

- `HomeController.php` - Gère la page d'accueil
- `ProductController.php` - Gère les produits et panier
- `ContactController.php` - Gère la page de contact

Modèles:`app/Models/`

- `Product.php` - Représente un produit
- `Cart.php` - Gère la session du panier

Vues:`app/Views/`

- `layout.php` - Modèle HTML global
- `home.php` - Contenu de l'accueil
- `products.php` - Contenu des produits
- `contact.php` - Contenu du contact

Flux Général Appliqué à Tous les Contrôleurs **1. Requête HTTP**

GET/POST vers une URL du site

→ **2. Routeur Slim**

`index.php` compare l'URL avec les routes enregistrées

→ **3. Instanciation du Contrôleur**

Création d'un objet du contrôleur approprié

→ **4. Appel de la Méthode**

Exécution de la méthode du contrôleur

→ **5. Gestion de la Vue**

Rendu HTML via Output Buffering (`ob_*`)

→ **6. Réponse HTTP**

Envoi du HTML au navigateur avec les headers appropriés

★ **2. Action 1: Afficher la Page d'Accueil**

Accueil - Page Principale

GET

GET /

Cheminement Détaillé

→ 1. Requête du Navigateur

```
GET http://localhost/
```

→ 2. Routage (index.php, ligne 35-37)

```
$app->get('/', function (Request $request,  
Response $response) { ... })
```

→ 3. Instanciation du Contrôleur

```
$controller = new HomeController();
```

→ 4. Appel de la Méthode

```
$controller->index($request, $response)
```

→ 5. Dans HomeController::index()

- Initialise \$pageTitle = 'Accueil - Épicerie BIO NATURE'
- Commence le buffering: ob_start()
- Inclut app/Views/home.php
- Récupère le contenu: \$mainContent = ob_get_clean()

→ 6. Rendu du Layout

- Nouveau buffering: ob_start()
- Inclut app/Views/layout.php
- Le layout utilise \$mainContent et \$pageTitle
- Récupère le HTML final: \$output = ob_get_clean()

→ 7. Envoi de la Réponse

```
$response->getBody()->write($output);  
return $response->withHeader('Content-Type',  
    'text/html; charset=utf-8');
```

Résumé de l'Action

- ✓ Affiche la page d'accueil du site avec informations sur l'épicerie
- ✓ Pas d'interaction avec la base de données ou session
- ✓ Rendu simple et direct de la vue HTML



3. Action 2: Afficher la Catégorie Produits

Liste des Produits

GET

GET /products

Cheminement Détaillé

→ 1. Requête du Navigateur

```
GET http://localhost/products
```

→ 2. Routage (index.php, ligne 39-42)

```
$app->get('/products', function (Request $request, Response $response) { ... })
```

→ 3. Instanciation du Contrôleur

```
$controller = new ProductController();
```

→ 4. Appel de la Méthode

```
$controller->index($request, $response)
```

→ 5. Dans ProductController::index()

- Récupère tous les produits:

```
$products = Product::getAllProducts();
```
- Crée une instance du panier:

```
$cart = new Cart();
```
- Initialise \$pageTitle = 'Produits - Épicerie BIO NATURE'

→ 6. Modèle Product - getAllProducts()

```
Retourne un tableau de 10 objets Product (5 fruits + 5 légumes)  
Fruits: Pommes, Bananes, Oranges, Fraises, Raisins  
Légumes: Tomates, Carottes, Laitue, Poivrons, Brocoli
```

→ 7. Rendu de la Vue

- `ob_start()`
- Inclut `app/Views/products.php`
- La vue utilise `$products` et `$cart`
- `$mainContent = ob_get_clean()`

→ 8. Rendu du Layout

- `ob_start()`
- Inclut `app/Views/layout.php`
- `$output = ob_get_clean()`

→ 9. Envoi de la Réponse

```
$response->getBody()->write($output);  
return $response->withHeader('Content-Type',  
    'text/html; charset=utf-8');
```

Résumé de l'Action

- ✓ Affiche une liste complète de 10 produits
- ✓ Initialise le panier dans la session
- ✓ Données en mémoire (pas de base de données)
- ✓ Affiche les prix et descriptions des produits
- ✓ Chaque produit a un bouton pour l'ajouter au panier

+ 4. Action 3: Ajouter un Produit au Panier

Ajouter au Panier

POST

POST /products/add/{id}

Exemple: POST /products/add/1 (ajouter le produit avec l'ID 1)

Cheminement Détaillé

→ 1. Requête du Navigateur

Formulaire HTML avec POST

```
form method="POST" action="/products/add/1"
```

→ 2. Routage (index.php, ligne 44-47)

```
$app->post('/products/add/{id}', function  
(Request $request, Response $response, array  
$args) { ... })
```

→ 3. Récupération de l'ID

\$args['id'] est extrait de l'URL

Exemple: /products/add/3 → \$args['id'] = '3'

→ 4. Instanciation du Contrôleur

```
$controller = new ProductController();
```

→ 5. Appel de la Méthode

```
$controller->addToCart($request, $response,  
$args)
```

→ 6. Dans ProductController::addToCart()

- \$productId = (int)\$args['id'];
- \$cart = new Cart();
- \$cart->addProduct(\$productId);

(Appelle le modèle Cart)

→ 7. Dans Cart::addProduct()

- `$product = Product::getProductById($productId);`
- Vérifie que le produit existe
- Ajoute ou augmente la quantité dans `$_SESSION['grocery_cart']`

Stockage en Session:

```
$_SESSION['grocery_cart'][1] = ['product' => Product Object,  
'quantity' => 1];
```

→ 8. Redirection

```
return $response->withStatus(302)-  
>withHeader('Location', '/products');  
(Redirection HTTP 302 vers la page produits)
```

Résumé de l'Action

- ✓ Ajoute ou augmente la quantité d'un produit au panier
- ✓ Utilise la session PHP pour persister les données du panier
- ✓ Valide l'existence du produit avant l'ajout
- ✓ Redirige l'utilisateur vers la page produits après l'ajout
- ✓ Le panier persiste entre les pages grâce à la session

— 5. Action 4: Retirer un Produit du Panier

Retirer du Panier

POST

```
POST /products/remove/{id}
```

Exemple: POST /products/remove/1 (retirer le produit avec l'ID 1)

Cheminement Détaillé

→ 1. Requête du Navigateur

Formulaire HTML avec POST

```
form method="POST" action="/products/remove/1"
```

→ 2. Routage (index.php, ligne 49-52)

```
$app->post('/products/remove/{id}', function  
(Request $request, Response $response, array  
$args) { ... })
```

→ 3. Récupération de l'ID

\$args['id'] contient l'ID du produit à retirer
Exemple: /products/remove/3 → \$args['id'] = '3'

→ 4. Instanciation du Contrôleur

```
$controller = new ProductController();
```

→ 5. Appel de la Méthode

```
$controller->removeFromCart($request, $response,  
$args)
```

→ 6. Dans ProductController::removeFromCart()

```
• $productId = (int)$args['id'];  
• $cart = new Cart();  
• $cart->removeProduct($productId);  
(Appelle le modèle Cart)
```

→ 7. Dans Cart::removeProduct()

```
unset($_SESSION['grocery_cart'][$productId]);  
(Supprime complètement le produit du panier)
```

 Effet sur la Session:

Le produit avec l'ID spécifié est supprimé du panier.
S'il y a plusieurs quantités, tout est supprimé (pas de gestion partielle).

 8. Redirection

```
return $response->withStatus(302) -  
>withHeader('Location', '/products');  
(Redirection HTTP 302 vers la page produits)
```

Résumé de l'Action

- ✓ Supprime complètement un produit du panier
- ✓ Utilise unset() pour supprimer l'entrée de la session
- ✓ Ne supprime que la quantité, le produit disparaît du panier
- ✓ Redirige vers la page produits
- ✓ Similaire à addToCart mais avec opération inverse

**6. Action 5: Vider le Panier Complètement**

Vider le Panier

POST

POST /clear-cart

Cheminement Détaillé

→ 1. Requête du Navigateur

Bouton ou lien dans le formulaire HTML
`form method="POST" action="/clear-cart"`

→ 2. Routage (index.php, ligne 54-58)

```
$app->post('/clear-cart', function (Request  
$request, Response $response) { ... })
```

→ 3. Instanciation du Modèle

```
$cart = new Cart();  
(Pas de contrôleur, ligne de route directe)
```

→ 4. Appel de la Méthode

```
$cart->clearCart();
```

→ 5. Dans Cart::clearCart()

```
$_SESSION['grocery_cart'] = [];  
(Établis le panier à un tableau vide)
```

Effet sur la Session:

Tous les produits sont supprimés du panier.

La clé 'grocery_cart' existe toujours mais contient un tableau vide.

→ 6. Redirection

```
return $response->withStatus(302)-  
>withHeader('Location', '/products');  
(Redirection HTTP 302 vers la page produits)
```

Résumé de l'Action

- ✓ Vide complètement le panier en une seule action
- ✓ Route directe sans contrôleur ou vue
- ✓ Réinitialise la session du panier
- ✓ Redirige vers la page produits
- ✓ Plus rapide que retirer chaque produit individuellement



7. Action 6: Afficher la Page de Contact

Page de Contact

GET

GET /contact

Cheminement Détaillé

→ 1. Requête du Navigateur

```
GET http://localhost/contact
```

→ 2. Routage (index.php, ligne 72-75)

```
$app->get('/contact', function (Request $request,
Response $response) { ... })
```

→ 3. Instanciation du Contrôleur

```
$controller = new ContactController();
```

→ 4. Appel de la Méthode

```
$controller->index($request, $response)
```

→ 5. Dans ContactController::index()

- Initialise \$pageTitle = 'Contact - Épicerie BIO NATURE'
- Récupère le tableau CONTACT_INFO (constante de classe)

→ 6. Données de Contact

Le contrôleur expose les informations:

- Nom: Épicerie BIO NATURE
- Email: contact@epiceribio.fr
- Téléphone: +33 1 23 45 67 89
- Adresse: 123 Rue du Marché, 75000 Paris
- Horaires: Lun-Ven 8h-19h, Sam 9h-18h, Dim 10h-12h30

→ 7. Rendu de la Vue

- `ob_start()`
- `Inclut app/Views/contact.php`
- La vue utilise `$pageTitle` et `$contactInfo`
- `$mainContent = ob_get_clean()`

→ 8. Rendu du Layout

- `ob_start()`
- `Inclut app/Views/layout.php`
- `$output = ob_get_clean()`

→ 9. Envoi de la Réponse

```
$response->getBody()->write($output);  
return $response->withHeader('Content-Type',  
'text/html; charset=utf-8');
```

Résumé de l'Action

- ✓ Affiche les coordonnées de contact de l'épicerie
- ✓ Données statiques stockées dans le contrôleur
- ✓ Pas d'interaction avec l'utilisateur
- ✓ Affiche téléphone, email, adresse et horaires
- ✓ Formulaire de contact (peut être présent dans la vue)



8. Action 7: Envoyer un Message de Contact

Envoyer Message (Placeholder)

POST

POST /send-contact

Cheminement Détaillé

→ 1. Requête du Navigateur

Formulaire HTML depuis la page contact
`form method="POST" action="/send-contact"`

→ 2. Routage (index.php, ligne 77-82)

```
$app->post('/send-contact', function (Request  
$request, Response $response) { ... })
```

→ 3. Traitement des Données

⚠ Fonction Placeholder (non implémentée)
Les données du formulaire peuvent être récupérées
via:
`$data = $request->getParsedBody();`
Mais aucun traitement n'est effectué actuellement

→ 4. Fonction Vide

Aucun email n'est envoyé
Aucune validation des données
Aucune sauvegarde en base de données
(À implémenter ultérieurement)

→ 5. Redirection

```
return $response->withStatus(302)-  
>withHeader('Location', '/contact');  
(Redirige vers la page contact sans confirmation)
```

⚠ État Actuel: Placeholder

- ✓ Fonction non implémentée
- ✓ Aucune validation des données du formulaire

- ✓ Aucun envoi d'email
- ✓ Aucune sauvegarde des messages
- ✓ À développer pour une utilisation réelle

Implémentation Recommandée

- ✓ Créer un contrôleur ContactController avec une méthode sendMessage()
- ✓ Valider les données du formulaire (nom, email, message)
- ✓ Utiliser une bibliothèque de mail (PHPMailer, SwiftMailer)
- ✓ Optionnel: Créer un modèle Message pour sauvegarder en base de données
- ✓ Ajouter un message de confirmation à l'utilisateur

✓ 9. Action 8: Valider la Commande

Validation de Commande (Placeholder)

GET

GET /validate-order

Cheminement Détaillé

→ 1. Requête du Navigateur

GET http://localhost/validate-order
Ou redirection depuis une page de paiement

→ 2. Routage (index.php, ligne 84-89)

```
$app->get('/validate-order', function (Request $request, Response $response) { ... })
```

→ 3. Traitement

⚠ Fonction Placeholder (simple)
Aucune validation réelle de commande
Pas d'interaction avec le panier

→ 4. Génération de la Réponse

```
$response->getBody()->write('...');  
Écrit directement un HTML simple:  
<h1>Commande validée !</h1>  
<p>Merci pour votre achat.</p>
```

→ 5. Envoi de la Réponse

```
return $response->withHeader('Content-Type',  
'text/html');  
(Répond avec un HTML minimaliste)
```

⚠ État Actuel: Placeholder Minimal

- ✓ Pas de contrôleur ou vue associé
- ✓ HTML écrit directement dans la route
- ✓ Aucune validation de commande
- ✓ Aucune interaction avec le panier

- ✓ Aucune gestion de paiement
- ✓ À implémenter pour un système de commande réel

Implémentation Recommandée

- ✓ Créer un modèle Order pour représenter une commande
- ✓ Vérifier que le panier n'est pas vide
- ✓ Créer une table en base de données pour les commandes
- ✓ Sauvegarder les articles du panier avec la commande
- ✓ Vider le panier après validation
- ✓ Intégrer un système de paiement (Stripe, PayPal)
- ✓ Créer une vue pour afficher l'échéance et confirmation

>



10. Modèles de Données



Modèle: Product

Fichier: `app/Models/Product.php`

Description: Représente un produit de l'épicerie

Propriétés Privées:

```
int $id - Identifiant unique du produit

string $name - Nom du produit

string $category - Catégorie (Fruits/Légumes)

float $price - Prix en euros

string $description - Description du produit
```

Méthodes Publiques:

```
__construct(int, string, string, float, string) -  
Constructeur  
  
getId(): int - Retourne l'ID  
  
getName(): string - Retourne le nom  
  
getCategory(): string - Retourne la catégorie  
  
getPrice(): float - Retourne le prix  
  
getDescription(): string - Retourne la description
```

Méthodes Statiques:

```
getAllProducts(): array - Retourne les 10 produits  
  
getProductsByCategory(string): array - Filtre par catégorie  
  
getProductById(int): ?Product - Récupère un produit par ID
```

Produits Disponibles:

ID	Nom	Catégorie	Prix
1	Pommes	Fruits	2,50€
2	Bananes	Fruits	1,80€
3	Oranges	Fruits	3,00€
4	Fraises	Fruits	4,50€
5	Raisins	Fruits	3,50€
6	Tomates	Légumes	2,80€
7	Carottes	Légumes	1,50€
8	Laitue	Légumes	2,00€

ID	Nom	Catégorie	Prix
9	Poivrons	Légumes	3,20€
10	Brocoli	Légumes	3,80€

Modèle: Cart

Fichier: `app/Models/Cart.php`

Description: Gère le panier de l'utilisateur stocké en session

Constantes:

```
SESSION_KEY = 'grocery_cart' - Clé de session pour le panier
```

Constructeur:

```
__construct() - Initialise la session du panier s'il  
n'existe pas
```

Méthodes Publiques:

```
addProduct(int, int = 1): void - Ajoute/incrémente un  
produit
```

```
removeProduct(int): void - Supprime complètement un produit
```

```
updateQuantity(int, int): void - Met à jour la quantité d'un  
produit
```

```
getCartItems(): array - Retourne tous les articles du panier
```

```
getCartTotal(): float - Calcule le total du panier
```

```
getItemCount(): int - Compte le nombre total d'articles
```

```
clearCart(): void - Vide complètement le panier
```

```
isEmpty(): bool - Vérifie si le panier est vide
```

Stockage en Session:

```
$_SESSION['grocery_cart'] = [ 1 => ['product' => Product, 'quantity'
=> 2], 3 => ['product' => Product, 'quantity' => 1], 7 => ['product'
=> Product, 'quantity' => 3] ]
```

Exemple d'Utilisation:

```
$cart = new Cart(); $cart->addProduct(1, 2); // Ajoute 2 pommes
$cart->addProduct(3); // Ajoute 1 orange $total = $cart-
>getCartTotal(); // Récupère le total $items = $cart->getCartItems();
// Récupère tous les articles
```



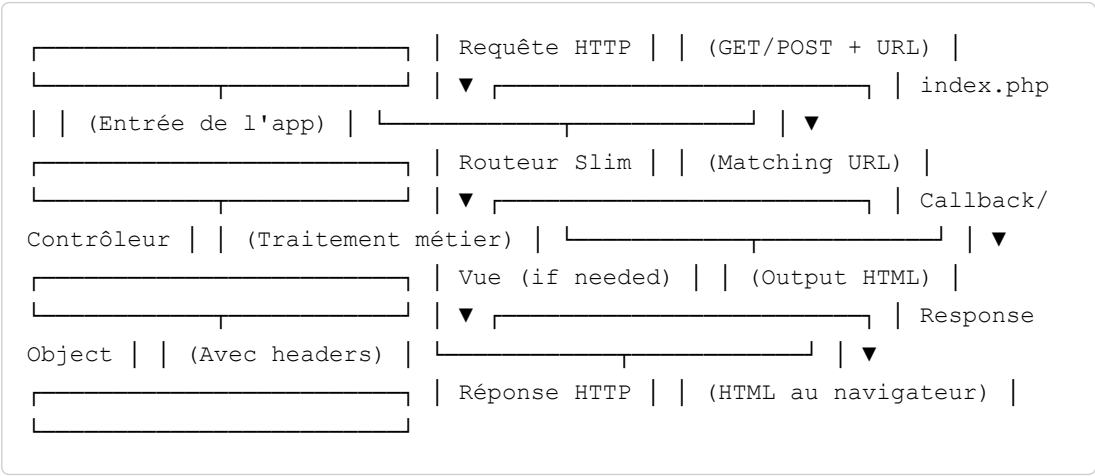
11. Résumé Technique

Routes Enregistrées

Méthode	URL	Contrôleur/Action	Type
GET	/	HomeController::index()	Page
GET	/products	ProductController::index()	Page
POST	/products/add/{id}	ProductController::addToCart()	Action
POST	/products/remove/{id}	ProductController::removeFromCart()	Action
POST	/clear-cart	Cart::clearCart()	Action
GET	/contact	ContactController::index()	Page

Méthode	URL	Contrôleur/Action	Type
POST	/send-contact	Placeholder	Action
GET	/validate-order	Placeholder	Page

Flux Général d'une Requête HTTP



Technologies Utilisées

Framework: Slim Framework 4 (PSR-7, PSR-15)

Langage: PHP 7.4+

Architecture: MVC (Model-View-Controller)

Pattern de Routage: FastRoute

Gestion de Session: Native PHP (\$_SESSION)

Base de Données: Aucune (données en mémoire pour produits)

Gestion de la Session

- ✓ La session est démarrée automatiquement dans le constructeur de Cart
- ✓ Les données du panier sont persistées dans `$_SESSION['grocery_cart']`
- ✓ La session existe tant que le cookie de session existe
- ✓ Le panier persiste entre les pages grâce à la session
- ✓ À la fermeture du navigateur, la session peut être perdue (paramètres PHP)

Améliorations Futures à Considérer

- ✓ Implémenter une base de données réelle (MySQL, PostgreSQL)
- ✓ Créer un système d'authentification utilisateur
- ✓ Implémenter les fonctions placeholder (contact, validation)

- ✓ Ajouter la gestion complète des commandes
- ✓ Intégrer un système de paiement
- ✓ Créer une interface d'administration
- ✓ Ajouter des tests unitaires et d'intégration
- ✓ Mettre en place une validation des formulaires robuste

Ce rapport a été généré le 23 février 2026.
Pour plus d'informations, veuillez consulter la documentation du projet.
Version du site: Épicerie BIO NATURE - SlimFramework